

Module 6: Transactions & Concurrency

Same story throughout: student **Aisha** (roll 101) takes **DBMS 101**, taught by **Prof. Rao**, and gets grade **A**. This module adds one detail you met in 5.5 — Aisha owes **12,000** in fees.

6.1 Transactions & ACID — Core • 14 min

Hook

Aisha pays her fees online. The database does two things: takes 12,000 off her balance, and adds 12,000 to the college account.

Between those two steps, the power cuts.

Her money is gone. The college never received it. Nobody did anything wrong.

Why + What

Real work is rarely one statement. A payment is two updates. Booking a seat is three. And a half-finished job is often worse than a job never started — as Aisha just discovered.

So the database needs a way to treat several statements as **one indivisible unit**.

A transaction is a group of statements that the database treats as a single unit of work — it either all happens, or none of it does.

The guarantees a transaction gives you are remembered as **ACID**.

How It Works (with the easiest example)

Aisha's payment, written properly:

```
sql
```

```
BEGIN;
```

```
UPDATE Student SET fees_due = fees_due - 12000 WHERE roll_no = 101;
```

```
UPDATE College SET cash = cash + 12000;
```

```
COMMIT;
```

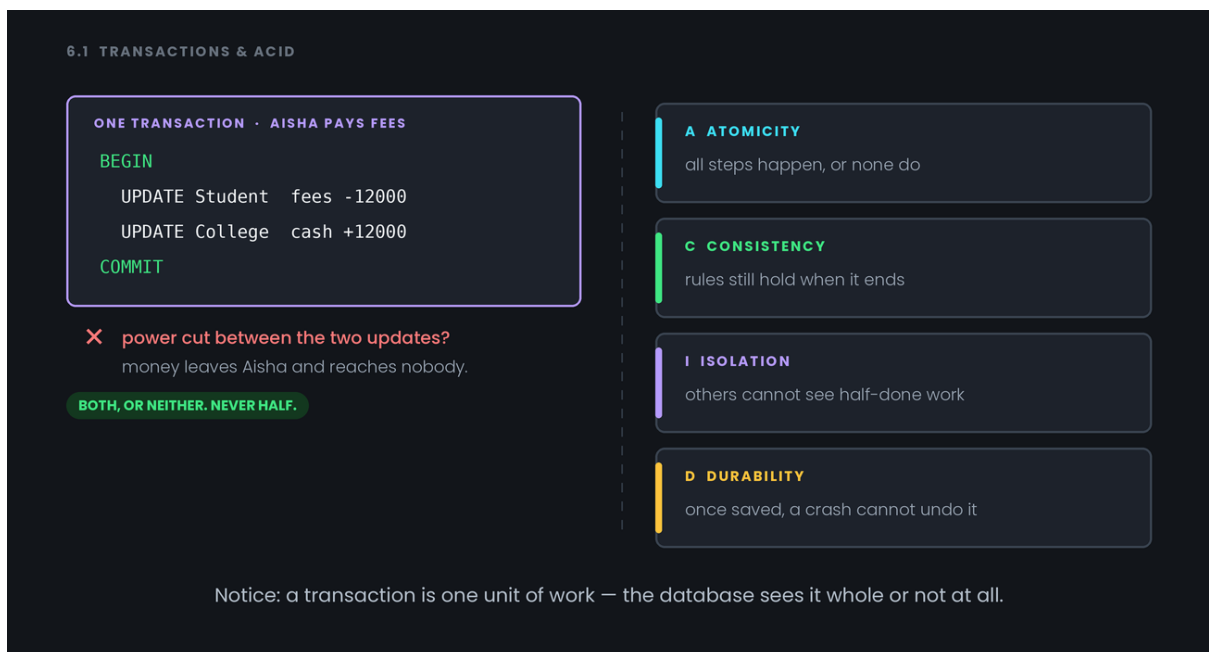
1. **BEGIN** — from here on, nothing is permanent yet. The database is keeping notes.
2. **The two updates run.** Anyone else looking right now still sees the **old** values. The work is invisible until it's finished.
3. **COMMIT** — both changes become real together. This is the moment they exist for everyone else.
4. **If anything fails before COMMIT** — a crash, an error, or an explicit ROLLBACK — the database throws away **both** updates. Aisha's balance goes back to 12,000. Nothing half-done survives.
5. **Name the four guarantees:**
 - **Atomicity** — all steps happen, or none do. This is what saves Aisha.

- **Consistency** — the database obeys its rules (Module 3's constraints) before and after.
- **Isolation** — other transactions can't see your half-finished work.
- **Durability** — once COMMIT returns, a crash cannot undo it.

Pause here. Which letter of ACID is broken if the power cut leaves Aisha's money deducted and the college unpaid?

Atomicity. Only part of the unit survived.

Visualize



Notice: a transaction is one unit of work — the database sees it whole or not at all.

Common Confusion

Atomicity and consistency get mixed up constantly. Split them like this:

- **Atomicity** is about the **steps** — did all of them happen, or none?
- **Consistency** is about the **rules** — is the database still valid at the end?

A transaction can be perfectly atomic and still be refused for consistency, if what it tried to do broke a foreign key.

Second: **durability doesn't mean "written to disk instantly."** It means the database has done enough that a crash can't lose the change — usually by writing a log entry first, then updating the actual data later. You'll see why in Module 7.

Interview Angle

1. "Explain ACID." — Give one line each **plus** one example. The payment example covers atomicity and isolation naturally, which is more convincing than four definitions.

2. *"What's the difference between COMMIT and ROLLBACK?"* — COMMIT makes every change in the transaction permanent; ROLLBACK discards all of them and restores the earlier state.
3. *"Give a real example where atomicity matters."* — Any two-sided money movement: deduct here, add there. Half of it is corruption, not a partial success.

Quiz: skipped — reinforced above.

Recap: A transaction is one unit of work that either fully happens or fully doesn't, and ACID names its four guarantees — atomicity, consistency, isolation, durability.

6.2 Concurrency Problems — Core • 12 min

Hook

One transaction at a time is easy. But a college portal has 900 students clicking at once on results day.

The database is now running many transactions **at the same time**, touching the same rows.

Four specific things can go wrong. They have names, and interviewers ask for them by name.

Why + What

You could run transactions strictly one after another and be perfectly safe. You'd also be unbearably slow — one student's slow query would block 899 others.

So databases overlap transactions to stay fast. Overlapping is where the trouble starts.

A concurrency problem is a wrong result caused by two transactions overlapping in time. The four to know are dirty read, lost update, non-repeatable read, and phantom read.

How It Works (with the easiest example)

Two transactions, T1 and T2, both touching Aisha's fees_due of 12,000.

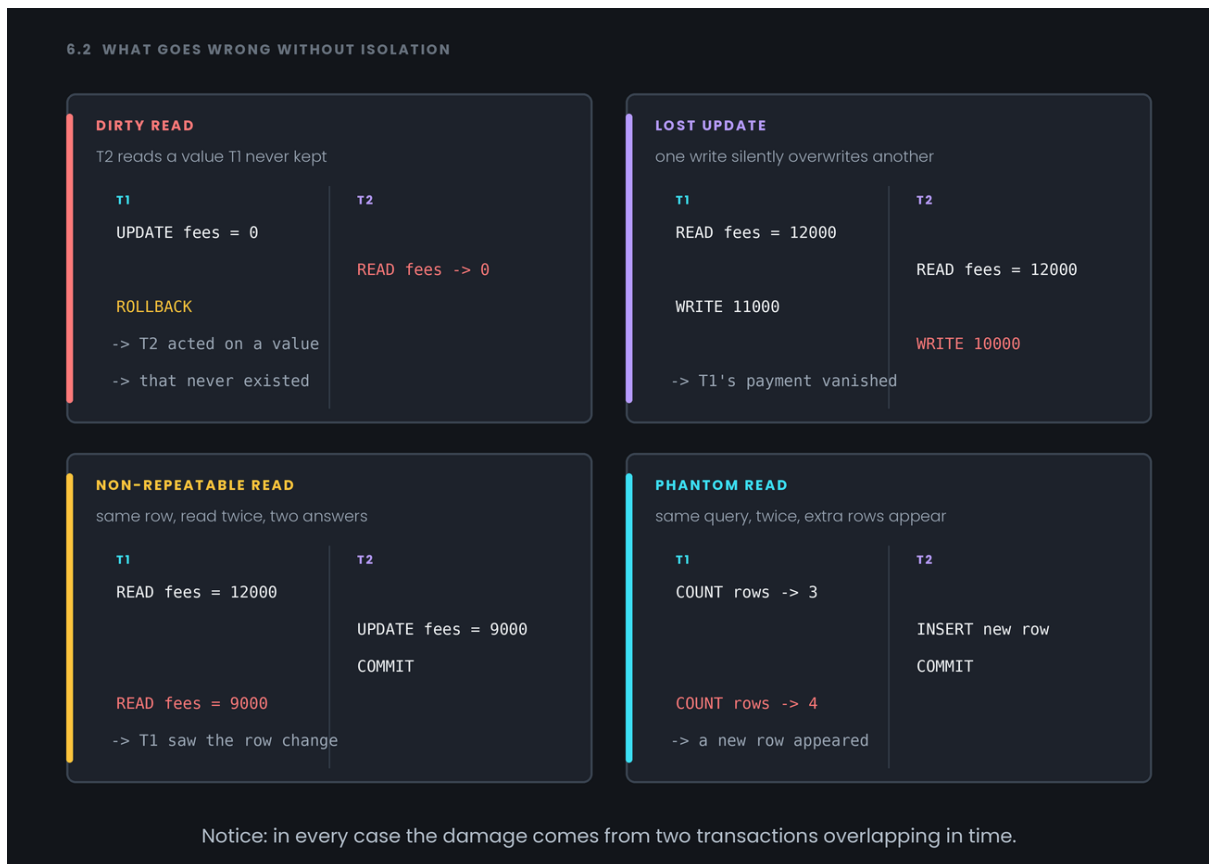
1. **Dirty read — reading something that never really existed.** T1 sets fees to 0. T2 reads it and sees 0. T1 then rolls back. T2 acted on a value that was never committed. It sent Aisha a "fees cleared" email for a payment that didn't happen.
2. **Lost update — one write silently erases another.** T1 reads 12,000. T2 reads 12,000. T1 subtracts 1,000 and writes 11,000. T2 subtracts 2,000 from *its* copy and writes 10,000. Both payments happened, but only one is reflected. **T1's 1,000 has vanished.**
3. **Non-repeatable read — same row, two different answers.** T1 reads fees = 12,000. T2 updates it to 9,000 and commits. T1 reads the **same row** again and now sees 9,000. Inside one transaction, a fact changed underneath it.
4. **Phantom read — same query, new rows appear.** T1 counts enrollments in DBMS 101 and gets 3. T2 inserts a new enrollment and commits. T1 runs the identical count and gets 4. No row T1 read changed — a **new** one showed up.

5. **Spot the pattern.** Non-repeatable read is about a **row that changed**. Phantom read is about **rows that appeared**. That single distinction is the most common exam question in this topic.

Pause here. T1 runs `SELECT COUNT(*)` twice and gets 5 then 6. Which problem is that?

Phantom read — the count changed because a new row appeared, not because an existing row was edited.

Visualize



Notice: in every case the damage comes from two transactions overlapping in time.

Common Confusion

Non-repeatable read vs phantom read is the classic trap. Use this test:

- Did a row you already read **change**? → non-repeatable read
- Did a **new row** turn up in your result? → phantom read

Row edited versus row added.

Second: people assume a **dirty read** means corrupt data. It doesn't. The value was perfectly valid at the moment it was read — it just belonged to a transaction that later gave up. "Dirty" means *uncommitted*, not *broken*.

Interview Angle

1. *"Name the concurrency problems and give an example of each."* — Dirty read, lost update, non-repeatable read, phantom read. A two-line T1/T2 sequence per problem beats any definition.
2. *"Difference between a non-repeatable read and a phantom read?"* — One is an existing row changing; the other is a new row appearing. Say it in exactly that shape.
3. *"Why not just run everything one at a time?"* — It's perfectly safe and unusably slow, because one slow transaction blocks everyone behind it.

Quiz

Q1. T2 reads a value that T1 later rolls back. This is: (a) lost update (b) dirty read (c) phantom read (d) deadlock **Answer: (b)** — T2 read uncommitted data that ended up never existing.

Q2. Two transactions read the same balance, each subtract from it, and one deduction disappears. This is: (a) lost update (b) dirty read (c) non-repeatable read (d) phantom read **Answer: (a)** — the second write overwrote the first, based on a stale copy.

Recap: Overlapping transactions cause four named problems — dirty read (uncommitted data), lost update (one write erases another), non-repeatable read (a row changed), phantom read (a row appeared).

6.3 Isolation Levels — Core • 12 min

Hook

You now know four ways concurrency can hurt you.

Here's the uncomfortable part: **preventing all four costs speed.** Sometimes a lot of it.

So the database doesn't decide for you. You choose.

Why + What

Not every query needs perfect safety. A dashboard counting logins can tolerate a slightly stale number. A fee payment cannot tolerate a lost update, ever.

So SQL gives you a dial, from "fast and risky" to "slow and airtight."

An isolation level is a setting that decides which concurrency problems your transaction is protected from. Higher levels prevent more problems and run slower.

How It Works (with the easiest example)

Four levels, weakest first. Each one adds protection to the one above it.

1. **READ UNCOMMITTED** — you can read other transactions' uncommitted changes. **All four problems possible**, dirty reads included. Almost never used deliberately.
2. **READ COMMITTED** — you only ever read committed data, so **dirty reads are gone**. But if you read the same row twice, it may have changed in between. This is the **default in most databases**, including PostgreSQL, Oracle and SQL Server.

3. **REPEATABLE READ** — any row you've read is frozen for the rest of your transaction. **Non-repeatable reads gone.** But new rows matching your search can still appear, so phantoms remain possible. This is **MySQL/InnoDB's default**.
4. **SERIALIZABLE** — the strictest. The result must be as if transactions ran one after another. **All four problems gone**, including phantoms. Slowest, and most likely to make transactions block or abort.
5. **Choose by the cost of being wrong.** Aisha's fee payment → SERIALIZABLE or at least REPEATABLE READ, because a lost update is real money. A "students online right now" counter → READ COMMITTED is plenty.

Pause here. Your transaction sees a row change between two reads. Which level would you move up to?

REPEATABLE READ — that's exactly the problem it removes.

Visualize

	DIRTY READ	LOST UPDATE	NON-REPEAT	PHANTOM
READ UNCOMMITTED fastest, least safe	×	×	×	×
READ COMMITTED the common default	○	×	×	×
REPEATABLE READ rows you read stay put	○	○	○	×
SERIALIZABLE safest, slowest	○	○	○	○

× can still happen ○ prevented

Notice: every row down the list is safer and slower — that is the whole trade.

Notice: every row down the list is safer and slower — that is the whole trade.

Common Confusion

People think higher isolation is simply "better." It isn't free, and it isn't free in a way that shows up in production: higher levels hold locks longer, so more transactions wait, and SERIALIZABLE will sometimes **abort** your transaction and make you retry.

Second: **the defaults differ between databases.** PostgreSQL, Oracle and SQL Server default to READ COMMITTED. MySQL's InnoDB defaults to REPEATABLE READ. Knowing that one fact is a small thing that makes an answer sound experienced.

Third: **SERIALIZABLE doesn't mean "one at a time."** Transactions still overlap. It only guarantees the **result** matches some one-at-a-time order.

Interview Angle

1. *"List the isolation levels and what each prevents."* — Four levels from READ UNCOMMITTED to SERIALIZABLE, each removing one more problem. Naming which problem drops out at each step is the whole answer.
2. *"Which is the default?"* — Usually READ COMMITTED; MySQL's InnoDB uses REPEATABLE READ. Say the level you'd pick depends on how expensive a wrong answer is.
3. *"Why not always use SERIALIZABLE?"* — Locks held longer, more waiting, and transactions may be aborted and need retrying. It's a throughput cost, not a correctness one.

Quiz

Q1. Which level still allows phantom reads? (a) READ UNCOMMITTED only (b) REPEATABLE READ and below (c) SERIALIZABLE (d) none **Answer: (b)** — repeatable read freezes rows you read, but new rows can still appear.

Q2. The lowest level that prevents dirty reads is: (a) READ UNCOMMITTED (b) READ COMMITTED (c) REPEATABLE READ (d) SERIALIZABLE **Answer: (b)** — that's precisely what "read committed" means.

Recap: Isolation levels are a dial from fast-and-risky to slow-and-safe — READ UNCOMMITTED allows everything, READ COMMITTED stops dirty reads, REPEATABLE READ stops non-repeatable reads, SERIALIZABLE stops phantoms too.

6.4 Locking & Deadlocks — Core • 12 min

Hook

Isolation levels are the setting you choose. But something has to actually *enforce* them.

That something is locks. And locks have a failure mode of their own — one where two transactions wait for each other **forever**.

Why + What

To stop T2 from reading a row T1 is halfway through changing, the database must make T2 **wait**. A lock is how it does that.

A lock is a claim a transaction puts on a row so others can't interfere with it. A shared (read) lock allows other readers. An exclusive (write) lock allows nobody else.

How It Works (with the easiest example)

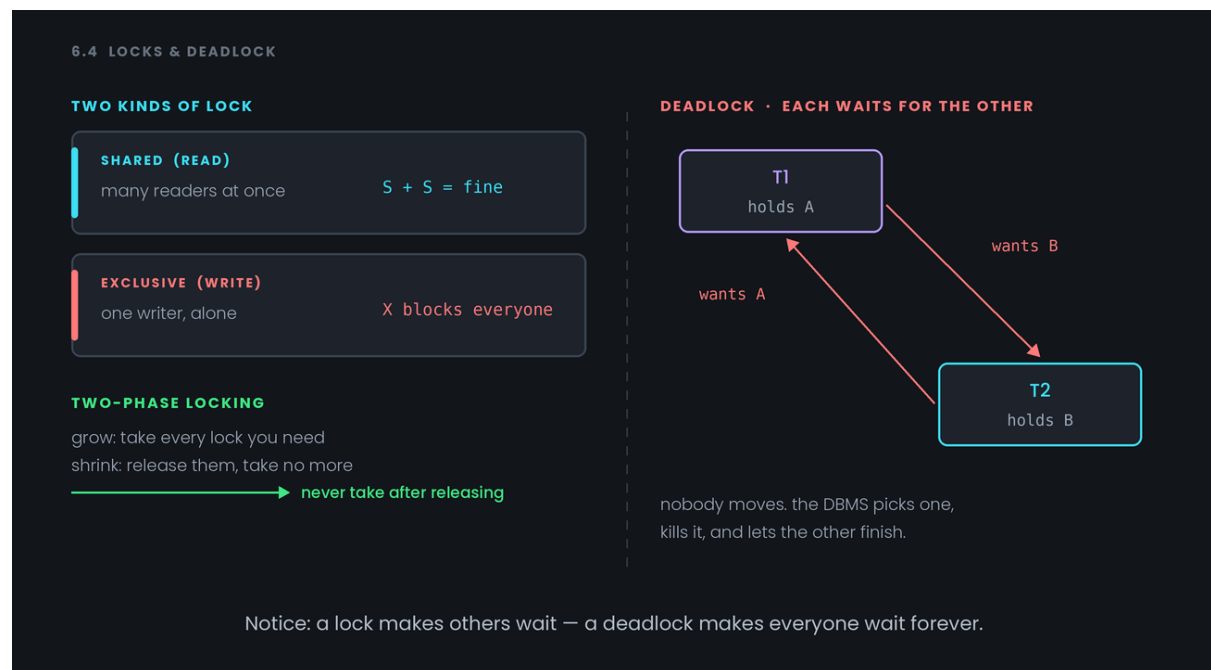
1. **Shared lock — reading.** T1 reads Aisha's fees and takes a shared lock. T2 wants to read the same row: allowed. **Many readers can hold shared locks together**, because reading changes nothing.
2. **Exclusive lock — writing.** T1 updates the fees and takes an exclusive lock. Now nobody else may read *or* write that row until T1 finishes. One writer, alone.

3. **Two-phase locking (2PL) — the rule that makes this safe.** Every transaction has a **growing** phase where it only takes locks, and a **shrinking** phase where it only releases them. **Once you've released one lock, you may never take another.** Follow that rule and the result is always equivalent to some serial order.
4. **Now the trap.** T1 locks row A and asks for row B. T2 locks row B and asks for row A. T1 waits for T2. T2 waits for T1. Neither will ever release, because neither can finish. That's a **deadlock**.
5. **How the database escapes.** It detects the cycle, picks a **victim**, kills that transaction and rolls it back. The survivor proceeds. Your application gets an error and retries — which is why real code wraps transactions in retry logic.

Pause here. Both transactions in your app update Student then College, in that order, every time. Can they deadlock on those two rows?

No. A deadlock needs the two transactions to grab the resources in **opposite** orders. Consistent ordering is the cheapest deadlock prevention there is.

Visualize



Notice: a lock makes others wait — a deadlock makes everyone wait forever.

Common Confusion

Deadlock and starvation are different failures.

- **Deadlock** — transactions wait for each other in a circle. Nobody ever moves.
- **Starvation** — one transaction *could* proceed, but keeps getting pushed to the back of the queue while others go first. It moves eventually, or never, but nothing is circular.

Circular wait versus bad luck in the queue.

Second: **a deadlock isn't a crash or a bug in the database.** It's an expected outcome of concurrent work. The database handles it by killing one side. Your job is to retry, and to reduce how often it happens — by touching tables in a consistent order and keeping transactions short.

Interview Angle

1. *"What is a deadlock, and how does a DBMS handle it?"* — Two or more transactions waiting on each other in a cycle. The DBMS detects it, kills a victim, rolls it back, and the other finishes. Add that the app should retry.
2. *"Difference between a shared and an exclusive lock?"* — Shared allows other readers; exclusive allows nobody else. Reads coexist, writes don't.
3. *"What is two-phase locking?"* — Take all locks in a growing phase, release in a shrinking phase, never take one after releasing. It guarantees serializability — but note that 2PL prevents nothing about deadlocks.
4. *"How would you reduce deadlocks in an application?"* — Always access tables in the same order, keep transactions short, and use the lowest isolation level the task actually needs.

Quiz

Q1. Two transactions each hold a row the other wants. This is: (a) starvation (b) deadlock (c) a dirty read (d) a phantom **Answer: (b)** — a circular wait, which is the definition of deadlock.

Q2. Under two-phase locking, a transaction may: (a) take and release locks in any order (b) take all locks, then release them, never taking again (c) only take shared locks (d) skip locks if the row is unchanged **Answer: (b)** — growing phase, then shrinking phase, no going back.

Recap: Locks enforce isolation — shared locks let readers share a row, exclusive locks give a writer sole access, 2PL orders lock taking and releasing, and a deadlock is a circular wait that the DBMS breaks by killing one transaction.

Module 7: Storage & Indexing

Same story throughout: student **Aisha** (roll 101) takes **DBMS 101**, taught by **Prof. Rao**, and gets grade **A**.

7.1 How Data Is Actually Stored — Core • 12 min

Hook

Your query works perfectly on 50 rows.

The college loads 5 years of real data — 2 million rows — and the same query now takes 40 seconds.

The SQL didn't change. So what did?

Why + What

Everything so far treated the database as an idea: tables, rows, keys. But rows live on a **disk**, and disks are staggeringly slow compared to memory.

That single fact explains almost every performance decision a database makes.

Data is stored on disk in fixed-size chunks called blocks (or pages). The database never reads one row — it reads the whole block that row sits in.

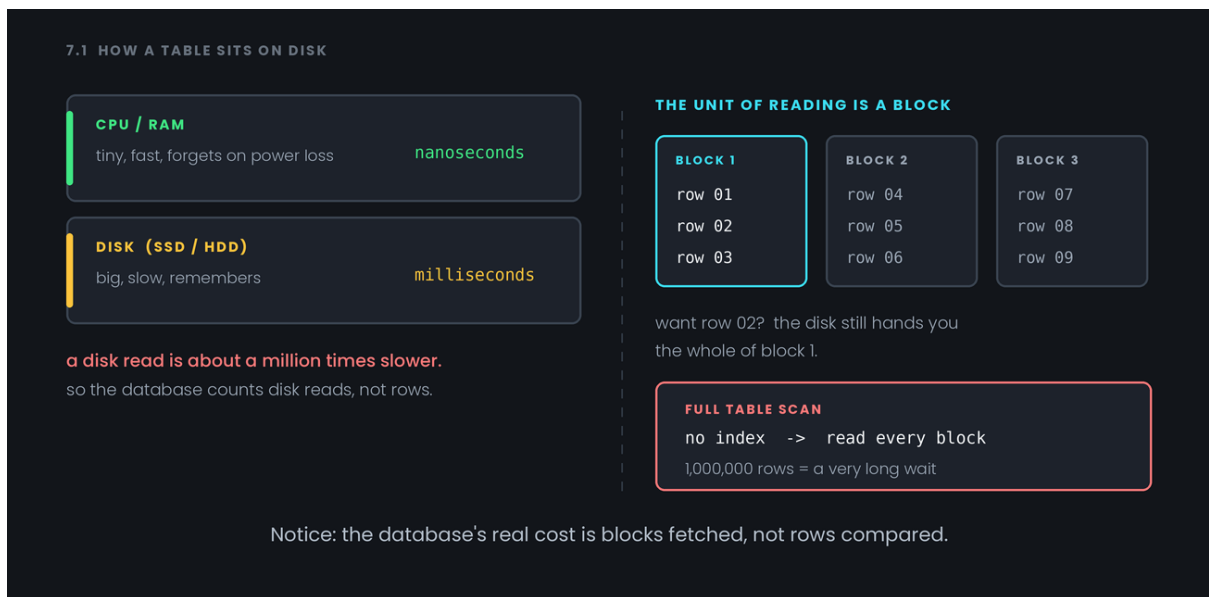
How It Works (with the easiest example)

1. **Two kinds of memory.** RAM is measured in nanoseconds and forgets everything on power loss. Disk is measured in milliseconds and remembers. A disk read is roughly **a million times slower** than a RAM read.
2. **So the database counts disk reads, not rows.** When it decides how to run your query, "how many rows do I compare" barely matters. "How many blocks must I fetch" is everything.
3. **Rows are packed into blocks.** A block is typically 4KB or 8KB. If a row is 100 bytes, roughly 80 rows fit in one block.
4. **You can't read half a block.** Ask for one row and the disk hands you the whole block it lives in. That's not waste — it's a bet that you'll want its neighbours next.
5. **Without help, the database reads everything.** WHERE roll_no = 103 with nothing to guide it means a **full table scan**: fetch block 1, check every row, fetch block 2, check every row, all the way to the end. 2 million rows is about 25,000 block reads.

Pause here. A table has 800,000 rows and 100 rows fit per block. Roughly how many block reads does a full table scan cost?

8,000. Rows divided by rows-per-block. That number — not the row count — is what determines how long you wait.

Visualize



Notice: the database's real cost is blocks fetched, not rows compared.

Common Confusion

People assume a faster CPU fixes slow queries. It usually doesn't. The CPU spends most of a slow query **waiting** for the disk. Adding cores to a query that's disk-bound changes almost nothing — which is why the fix is nearly always "read fewer blocks," not "compute faster."

Second: SSDs are much faster than spinning disks, and that helps a lot. But they're still far slower than RAM, so the block-counting logic is unchanged. SSDs make bad queries less painful, not fast.

Interview Angle

1. "Why does a query get slower as a table grows?" — More rows means more blocks, and without an index the database must fetch all of them. Say the cost scales with blocks read.
2. "What is a full table scan and when is it acceptable?" — Reading every block of a table. It's fine on small tables, or when you genuinely need most of the rows anyway — an index wouldn't help there.
3. "Why does the database read a whole block for one row?" — Disks transfer in fixed chunks, and neighbouring rows are often wanted next, so the block is the natural unit.

Quiz: skipped — reinforced above.

Recap: Data lives on disk in fixed-size blocks, disk reads are around a million times slower than memory, so a database's real cost is blocks fetched — and with no index, that means every block in the table.

7.2 Indexes: What They Are & How They Help — Core • 16 min

Hook

You need one definition from a 900-page textbook.

You could turn every page. Or you could flip to the index at the back, find the word, and go straight to page 412.

The database has the same choice.

Why + What

A full table scan is turning every page. It works, and it gets slower every time the table grows.

The fix is the same as the book's: keep a small, **sorted** list of one column, with a pointer to where each value actually lives.

An index is a separate structure that stores one or more columns in sorted order, along with a pointer to the row's location — so the database can jump straight to a value instead of scanning for it.

How It Works (with the easiest example)

The Student table stores rows in whatever order they were inserted: 104 Dev, 101 Aisha, 107 Gita, 103 Cara. Now find roll 103.

1. **Without an index.** The rolls are unsorted, so there's no way to skip ahead. Read every block. Cara happens to be last, so you did all the work.
2. **Build an index.**

sql

```
CREATE INDEX idx_roll ON Student(roll_no);
```

3. **What that actually created.** A second, small structure holding roll_no in **sorted** order — 101, 103, 104, 107 — each with a pointer to the block its row lives in.
4. **Now search again.** Because the index is sorted, the database can jump — check the middle, decide which half, repeat. It finds 103 in a couple of hops, reads the pointer, and fetches exactly **one** block of the real table.
5. **Notice why the primary key is already fast.** Databases build an index on the primary key automatically. That's why WHERE roll_no = 101 was never the slow query — you had an index all along without asking.

Two kinds are worth knowing:

- **Clustered index** — the **table itself** is kept in this order. The rows *are* the index, so there's no pointer hop. You get **one per table**, because rows can only be in one physical order.
- **Non-clustered index** — a separate sorted list that points back to the rows. You can have **many**, and each one costs extra disk space.

Pause here. Why can a table have only one clustered index?

Because a clustered index defines the physical order of the rows on disk, and rows can only be stored in one order at a time.

Visualize

7.2 WHAT AN INDEX IS

THE TABLE · STORED IN INSERT ORDER

BLOCK	ROLL_NO	NAME
1	104	Dev
1	101	Aisha
2	107	Gita
2	103	Cara

CREATE INDEX →

THE INDEX · SORTED, WITH POINTERS

ROLL_NO (SORTED)	POINTS TO
101	block 1
103	block 2
104	block 1
107	block 2

✗ unsorted - finding 103 means reading all

jump straight to 103 - one block read

CLUSTERED

the table itself is kept in this order

one per table · the rows ARE the index

NON-CLUSTERED

a separate sorted list that points back

many per table · costs extra space

Notice: an index is a second, sorted copy of one column - plus a pointer home.

Notice: an index is a second, sorted copy of one column — plus a pointer home.

Common Confusion

"So index everything?" No — and this is the answer interviewers are actually listening for.

Every index must be **kept up to date**. Every INSERT, UPDATE and DELETE on that column has to modify the index too. Ten indexes means ten extra updates per write. So an index buys you read speed and pays for it with **write speed and disk space**.

Second: **an index can be quietly ignored**. Write WHERE UPPER(name) = 'AISHA' and the index on name becomes useless, because the index stores names, not upper-cased names. Same with a leading wildcard: LIKE '%isha' can't use an index, though LIKE 'Ais%' can. Wrapping an indexed column in a function is the most common way people accidentally disable their own index.

Third: **an index doesn't help when you want most of the rows**. If your query returns 80% of the table, hopping through an index and fetching rows one at a time is *slower* than just scanning the whole thing. The optimiser knows this and will skip your index on purpose.

Interview Angle

1. "What is an index and how does it speed up a query?" — A sorted structure with pointers, letting the database jump to a value instead of scanning. Mention that it turns "read every block" into "read one or two."
2. "Why not index every column?" — Every write must update every index, and each index costs space. Reads get faster, writes get slower.

3. *"Difference between clustered and non-clustered?"* — Clustered sets the physical row order, so one per table and no pointer hop. Non-clustered is a separate list pointing back, so many per table.
4. *"Give a case where an index exists but isn't used."* — A function wrapping the column, a leading % wildcard, or a query returning most of the table.

Quiz

Q1. How many clustered indexes can a table have? (a) one (b) one per column (c) unlimited (d) none **Answer: (a)** — it defines the physical order, and rows can only sit in one order.

Q2. Adding many indexes to a heavily-written table mainly costs you: (a) read speed (b) write speed and disk space (c) correctness (d) nothing **Answer: (b)** — every insert and update must maintain every index.

Q3 (scenario). WHERE roll_no = 101 is fast, but WHERE UPPER(name) = 'AISHA' is slow even though name is indexed. Why, and what would you change? **Answer:** The index stores name as written, not upper-cased, so the database can't match against it and falls back to a full scan. Fixes: compare without the function (WHERE name = 'Aisha'), or create a **functional index** on UPPER(name) if the case-insensitive search is genuinely needed.

Recap: An index is a sorted structure with pointers that lets the database jump to a value instead of scanning — clustered sets the physical row order (one per table), non-clustered is a separate list (many per table), and every index makes writes slower.

7.3 B+ Trees & Hash Indexes — Core • 12 min

Hook

You know an index is "sorted with pointers." But *how* is it sorted, physically, when the table has 10 million rows and rows are added all day?

A plain sorted list would need reshuffling on every insert. Something smarter is needed.

Why + What

Almost every relational database uses the same structure, and it's worth being able to name it.

A B+ tree is a balanced tree where all the real data sits in the leaves, the leaves are linked to each other, and every search takes the same small number of steps.

There's also a second kind, good at exactly one thing:

A hash index turns the search value into a location directly — extremely fast for exact matches, and useless for ranges.

How It Works (with the easiest example)

Index the rolls 101, 102, 103, 104, 107, 109.

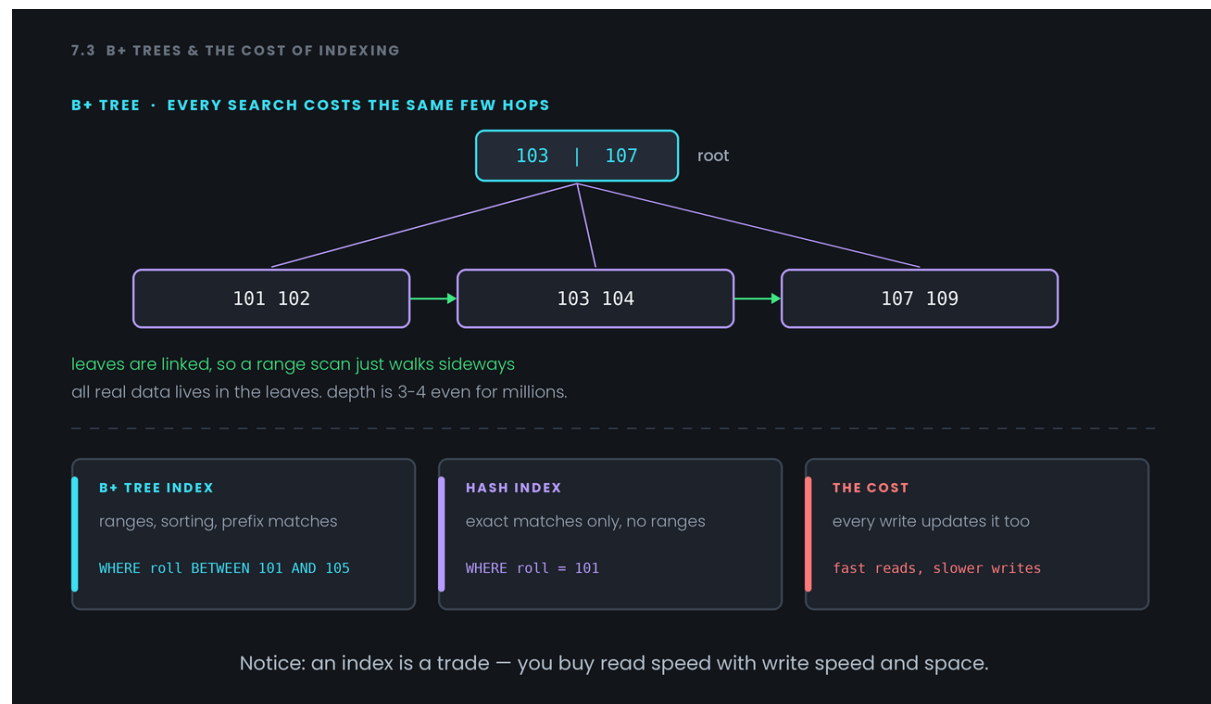
1. **The root holds signposts, not data.** It might say 103 | 107 — meaning "less than 103 goes left, 103 to 106 goes middle, 107 and up goes right." No actual rows here.

2. **The leaves hold everything.** All the real keys live at the bottom: 101 102, 103 104, 107 109. That's the "+" in B+ tree — data only in leaves, so the upper levels stay small.
3. **Every search costs the same.** Root → maybe one middle level → leaf. Because the tree is **balanced**, no key is deeper than any other. Even for millions of rows, depth is typically **3 or 4** — so a lookup is 3 or 4 block reads instead of 25,000.
4. **The leaves are linked sideways.** So WHERE roll BETWEEN 101 AND 105 finds 101, then simply **walks along the leaf chain** until it passes 105. That's why B+ trees are excellent at ranges, sorting and ORDER BY.
5. **A hash index works differently.** It runs the value through a hash function and lands directly on the location — one step, no tree walk. Brilliant for WHERE roll = 101. But hashing destroys order, so BETWEEN, >, < and ORDER BY get no help at all.

Pause here. Your query is WHERE grade > 'B'. Which index type helps — B+ tree or hash?

B+ tree. It's a range query, and hash indexes lose all ordering.

Visualize



Notice: an index is a trade — you buy read speed with write speed and space.

Common Confusion

B-tree vs B+ tree. In a B-tree, data can sit at any level, including the root. In a B+ tree, data lives **only in the leaves**, and those leaves are linked. That linking is exactly what makes range scans fast, which is why real databases use B+ trees. In casual speech people say "B-tree index" and mean B+ tree — understand the difference, don't correct the interviewer.

Second: **"balanced" isn't decoration.** It means the tree reorganises itself as rows are inserted so that all leaves stay at the same depth. Without it, an index built by inserting 1, 2, 3, 4... in order would degrade into a long chain — as slow as no index at all.

Third: **hash indexes are rarer than students expect.** They're not the default anywhere, because most real queries involve ranges, sorting or ORDER BY somewhere, and a hash index helps with none of it.

Interview Angle

1. *"Why do databases use B+ trees for indexes?"* — Balanced depth means every lookup costs the same few reads, and linked leaves make range scans and sorting fast.
2. *"Difference between a B-tree and a B+ tree?"* — B+ trees keep all data in the leaves and link them together; B-trees store data at every level. The linking is what makes ranges efficient.
3. *"When would a hash index beat a B+ tree?"* — Pure equality lookups with no ranges and no ordering. Say that it loses to B+ trees the moment BETWEEN or ORDER BY appears.
4. *"Roughly how many reads to find one row in an index over a million rows?"* — About 3 or 4, because the tree stays shallow. That contrast against a full scan is the whole point of the topic.

Quiz

Q1. In a B+ tree index, actual data pointers are stored: (a) in the root (b) at every level (c) only in the leaf nodes (d) outside the tree **Answer: (c)** — upper levels are signposts only, which keeps the tree shallow.

Q2. Which query gets no benefit from a hash index? (a) WHERE roll = 101 (b) WHERE roll BETWEEN 101 AND 105 (c) WHERE roll = 107 (d) all benefit equally **Answer: (b)** — hashing destroys ordering, so ranges can't use it.

Recap: B+ trees keep all data in linked leaves at equal depth, giving fast lookups and fast ranges in 3–4 reads, while hash indexes give one-step exact matches but no ranges or ordering at all.